

When an executable is run, the kernel acts as the initial loader. On Linux, this happens through the `execve` system call. The kernel reads the ELF header to locate the Program Header Table, then maps the required segments (such as code and data) into memory according to their offsets, virtual addresses, sizes, and permissions.

If a `PT_INTERP` segment is present, the kernel loads and transfers control to the dynamic loader (also called the dynamic linker), typically `/lib64/ld-linux-x86-64.so.2` on x86_64 systems.

The dynamic loader then processes the `PT_DYNAMIC` segment to:

- locate required shared libraries (`DT_NEEDED`)
- load them into memory using `mmap`
- resolve symbols (like `printf`, `puts`)
- apply relocations

Regarding how shared libraries are loaded:

The dynamic loader does **not load the entire library eagerly into physical memory**, nor does it load only a single function at a time. Instead, it maps the required segments of the shared library (via `mmap`) into the process's virtual address space.

Actual physical memory pages are loaded **on demand** using paging:

- When a function (e.g., `printf`) is called for the first time, a page fault occurs if its page is not yet in RAM.
- The kernel then loads the needed pages from disk into memory.

Additionally, symbol resolution can be:

- **Eager (immediate binding)**: all symbols are resolved at startup
- **Lazy binding (default)**: function addresses are resolved the first time they are called (via the PLT/GOT mechanism)

So:

- The loader maps the library into virtual memory (not just a pointer, and not just one function)
- The kernel loads actual pages into RAM only when accessed
- The dynamic loader resolves symbols either at startup or on first use

Finally, control is transferred to the program's entry point after all required setup is complete.